

Portada y presentación

Saludo, título del trabajo y tribunal.

🗣️ QUÉ DECIR

- ▶ Buenas tardes. Soy **Nicolás Rey Alonso**.
- ▶ Presento mi TFG: «**Framework backend–frontend para el desarrollo rápido de proyectos apoyado en un modelo de datos en evolución**».
- ▶ Aplicación a los proyectos con el **Jardín Botánico Viera y Clavijo (JBVC)**.
- ▶ Tutor de empresa: **Rafael Juan Nebot Medina**. Tutora académica: **María Dolores Afonso Suárez**.
- ▶ Desarrollado en colaboración con el **Instituto Tecnológico de Canarias (ITC)**.

🔑 CLAVES & ACLARACIONES

- Framework = dos artefactos: `uniback` + `ngt-gui`.
- Stack visible en portada: FastAPI · SQLAlchemy 2 · Continuum · JSON Schema · Angular · Formly · PostgreSQL · Docker.
- Hablar despacio aquí: marca el tono de toda la defensa.

→ Empiezo por el contexto y el problema.

Estructura de la presentación

La presentación recorre cinco bloques.

🗣️ QUÉ DECIR

- ▶ **I — Problema y objetivos** (ITC · JBVC · 5 OE).
- ▶ **II — Estado del arte** (stack y comparativa).
- ▶ **III — Arquitectura** (polimorfismo, plugins, capas).
- ▶ **IV — Cuatro incrementos** (API · contrato · Formly · bundle).
- ▶ **V — Resultados y conclusiones** (pruebas · antes/después).

🔑 CLAVES & ACLARACIONES

- Hilo conductor: «del problema al framework consolidado y validado».
- Recordar los dos nombres: `uniback` (backend) y `ngt-gui` (frontend).

Origen del proyecto

Del trabajo de prácticas a un caso de uso real.

🗣️ QUÉ DECIR

- ▶ El trabajo nace de mi **estancia de prácticas en el ITC**.
- ▶ Allí desarrollé **NextGenDem-GUI-LIB**, un catálogo de componentes Angular, pero **acoplado a un backend específico** → poco reutilizable.
- ▶ En paralelo, el ITC arranca el proyecto con el **JBVC**: catalogación, anotaciones y jerarquías taxonómicas → **modelo de datos en evolución constante**.
- ▶ Ese contexto convierte la **reutilización en una necesidad operativa**.

🔑 CLAVES & ACLARACIONES

- Estado al iniciar el TFG:
- • Frontend: librería propia (NGT) pero atada a NextGenDem por un servicio con **+270 métodos hardcoded**.
- • Backend: base del ITC con modelos ORM completos pero **sin capa de API homogénea** (ni envoltorio, ni fábricas, ni esquema).

→ ¿Por qué es un problema que el modelo cambie?

El problema

Cuando el modelo evoluciona, el coste se propaga en cascada.

🗣️ QUÉ DECIR

- ▶ Cada cambio en el modelo obliga a tocar **cinco capas**: ORM, validación/esquemas, endpoints, cliente HTTP y formularios.
- ▶ Buena parte de ese código se **escribe a mano** y se **acopla al dominio**.
- ▶ **Hipótesis del trabajo**: ese coste puede reducirse al mínimo si el modelo es la **única fuente de verdad** y el resto se **deriva automáticamente**.

🔑 CLAVES & ACLARACIONES

- Cita (§1.2): «Reducir al mínimo el código que hay que escribir y mantener para exponer, validar, comunicar y editar una entidad cuando el modelo está en evolución constante.»
- Enfoque propuesto: ORM → JSON Schema (x-*) → CRUD derivado → `EntityClient` → `<ngt-dynamic-form>` (un cambio se propaga solo).

→ La solución concreta.

Hipótesis y propuesta

Un framework full-stack con dos artefactos.

🗨️ QUÉ DECIR

- ▶ **uniback** (Python): núcleo de modelos reutilizables, envoltorio normalizado, fábricas CRUD y **derivación automática de JSON Schema enriquecido desde el ORM**.
- ▶ **ngt-gui** (Angular, librería publicable): consume el contrato con un **cliente genérico** y genera **formularios Formly sin HTML por entidad**.
- ▶ Idea central: **el ORM es la única fuente de verdad**; un cambio se propaga al esquema, al contrato y a la interfaz **automáticamente**.

🔑 CLAVES & ACLARACIONES

- uniback: FastAPI · SQLAlchemy 2.x · Continuum · Alembic · Pydantic · PostgreSQL · Redis+Celery.
- Entrada única: `initialize(config) → (orm_base, sessions, app)`.
- ngt-gui: Angular · Formly · ng-zorro · EntityClient · SchemaService · ng-packagr.
- Uso: `<ngt-dynamic-form entityPath="/roles">`.

Cinco objetivos específicos

Cinco objetivos específicos, todos completados.

QUÉ DECIR

- ▶ **OE1 — API modular:** envoltorio uniforme, catálogo de errores y fábricas CRUD.
- ▶ **OE2 — Modelado de datos:** consolidar el núcleo ORM con versionado e histórico.
- ▶ **OE3 — Comunicación estandarizada:** descubrimiento de entidades y cliente genérico tipado.
- ▶ **OE4 — Generador de formularios:** sobre Formly y Grid, guiado por el esquema.
- ▶ **OE5 — Generación de esquemas:** derivados automáticamente desde el ORM.

CLAVES & ACLARACIONES

- Los cinco quedan **COMPLETADOS** al cierre del trabajo y **respaldados por pruebas automatizadas**. (§2.4)
- Si preguntan: cada OE se mapea con un incremento del bloque IV.

Quién espera qué

Cuatro stakeholders, una sola pieza conectora.

🗣️ QUÉ DECIR

- ▶ **Propietario — ITC:** base reutilizable que reduzca el coste de arrancar y mantener nuevos proyectos.
- ▶ **Constructor — equipo de desarrollo:** necesita una API predecible, tipada y bien documentada.
- ▶ **Cliente / caso real — JBVC:** aporta el caso de uso que **valida la genericidad**.
- ▶ **Beneficiario — usuario final:** no toca el framework, pero recibe formularios y vistas correctas y validadas.

🔑 CLAVES & ACLARACIONES

- Matiz clave (§2.1.2): el framework **no es la solución final** para el Jardín, sino **la herramienta con la que esa solución se construye**.
- Útil para responder «¿y la aplicación del Jardín?»: es el primer consumidor, no el objeto del TFG.

Comparativa

El hueco: cerrar la cadena desde el ORM hasta el formulario.

🗣️ QUÉ DECIR

- ▶ **react-jsonschema-form** y **JSON Forms**: lado cliente, asumen un esquema ya dado.
- ▶ **OpenAPI / Swagger**: describe operaciones, no presentación.
- ▶ **Admin autogenerados** (p. ej. Django Admin): acoplan la interfaz al servidor, difíciles de reutilizar.
- ▶ **Aportación diferencial**: derivar el esquema del ORM, enriquecerlo con pistas, transportarlo por un **contrato versionado con ETag** y consumirlo con un **cliente genérico**.

🔑 CLAVES & ACLARACIONES

- Tabla, columnas: Desde ORM · Hints UI · Contrato · Versionado.
- Solo `uniback+ngt-gui` cubre las cuatro (x-*, contrato uniforme, ETag).
- Frase fuerte: la aportación **no es el renderizador**, es **cerrar la cadena** manteniendo back y front desacoplados por un contrato versionado.

Stack

Elegido para que cada capa hable el mismo idioma.

🔊 QUÉ DECIR

- ▶ **Backend:** FastAPI + Pydantic **emiten JSON Schema de forma natural**; SQLAlchemy 2 (API `Mapped[...]`) da introspección rica del modelo.
- ▶ **Continuum + Alembic:** versionado automático + migraciones reproducibles.
- ▶ **Postgres + Docker:** JSONB, full-text search, despliegue multiplataforma. **Redis + Celery:** asíncrono opcional.
- ▶ **Frontend:** Angular + Formly (config declarativa) + ng-zorro (base de ~20 tipos Formly propios), empaquetado con ng-packagr.

🔑 CLAVES & ACLARACIONES

- **Idioma común entre capas: JSON Schema · OpenAPI 3.x · extensiones** `x-*` . (§4.1–§4.2)
- Es el argumento de coherencia: no son piezas sueltas, comparten formato.

Flujo extremo a extremo

Del ORM al formulario, en una sola línea de flujo.

🗣️ QUÉ DECIR

- ▶ El **modelo SQLAlchemy** es la única fuente de verdad.
- ▶ `sqlalchemy_to_pydantic` genera el modelo Pydantic; `get_enriched_json_schema` produce el esquema con `x-*` **por columna**.
- ▶ Se publica en **tres endpoints**: `schema.json` por entidad, el **bundle** agregado y el **descubrimiento** de entidades.
- ▶ Todo viaja en un **envoltorio uniforme**; el front lo **cachea con ETag**, `schemaToFormly` lo traduce y `<ngt-dynamic-form>` lo renderiza.

🔑 CLAVES & ACLARACIONES

- Pistas: `x-ui-widget` · `x-options-endpoint` · `x-hide-when`.
- Envoltorio: `{ content, count, issues }`.
- Endpoints: `GET /<e>/schema.json` · `GET /api/sys/schemas [ETag]` · `GET /api/sys/entities`.
- Esta es la diapositiva-resumen: si me pierdo, vuelvo a este flujo.

Núcleo del modelo

FunctionalObject: tratar lo heterogéneo de manera homogénea.

🗣️ QUÉ DECIR

- ▶ Cada entidad de dominio **hereda de** `FunctionalObject` y se asocia a un `ObjectType` con código numérico.
- ▶ Eso habilita el **tratamiento polimórfico**: anotaciones, ACLs y colecciones se refieren a «cualquier objeto funcional» **sin conocer su clase concreta**.
- ▶ La **autorización** también usa herencia polimórfica: `Identity` y `Role` heredan de `Authorizable`.

🔑 CLAVES & ACLARACIONES

- Permisos por tipo o por instancia: `ACL` + `ACLExpression` + `ACLDetail` (a nivel de `ObjectType` o de `FunctionalObject` concreto).
- Cuando una clase es **raíz de jerarquía**, el esquema emite un `oneOf` con sus subclases y un `x-discriminator` deducido de `object_type_id`. (§6.5–§6.6)
- Esto conecta con el bundle (Incremento 4).

Extensibilidad · plugins

El dominio entra desde fuera: el núcleo no lo conoce.

🔊 QUÉ DECIR

- ▶ Un **plugin** es un módulo Python autónomo que hereda de `UnibackPlugin` y sobrescribe **solo los hooks que necesita** (modelos, routers, semilla).
- ▶ Más **cinco contratos de extensión inyectables**: adaptadores de origen, importadores/exportadores, widgets de UI y resolvers de FK.
- ▶ El `PluginManager` los **descubre solos al arrancar** (escanea `src/plugins`, sin registro manual); política «el último registrado gana».
- ▶ **Ejemplo — plugin de plantas**: declara `Plant` (subclase de `FunctionalObject`) y su CRUD; el contrato, el esquema y el formulario se derivan solos. **A mano, solo el mapa.**

🔑 CLAVES & ACLARACIONES

- Materializa el requisito de **reutilización (RNF-01)**: el núcleo no acopla lógica de dominio.
- Hooks: `on_init`, `get_model_modules`, `get_routers`, `on_seed`.
- Extensiones: `get_field_widgets`, `get_importers/exporters`, `get_fk_resolvers`. (§6.3)

API backend y modelado

Despliegue reproducible y núcleo de modelos consolidado.

🔊 QUÉ DECIR

- ▶ **Entorno reproducible** con `docker compose` (Postgres · Redis · worker Celery · web), probado en **Windows, macOS y Linux** con CPUs distintas.
- ▶ Modelos ORM organizados en **8 módulos de dominio**.
- ▶ **SQLAlchemy-Continuum** para versionado nativo (histórico automático).
- ▶ **Punto de entrada único** `initialize(config)` : permite usar `uniback` como **librería** en proyectos consumidores.

🔑 CLAVES & ACLARACIONES

- 8 módulos: core · sysadmin · annotations · hierarchies · species · files · geographics · screens.
- Restricción crítica: `make_versioned()` **antes** de definir los modelos.
- Prefijo de tablas configurable (`ub_` por defecto) → convive con tablas del consumidor sin colisión. (§7.3.1)

Contrato de comunicación

ResponseEnvelope: una sola forma, éxito o error.

🗣️ QUÉ DECIR

- ▶ **Toda respuesta** (éxito o error) adopta la misma forma: `content`, `count`, `issues`.
- ▶ Catálogo de **códigos estándar** (BAD_REQUEST, UNAUTHORIZED, VALIDATION_ERROR...) y **manejadores globales** que traducen cualquier excepción al envoltorio.
- ▶ Las fábricas `make_simple_rest_crud` y `make_crudie_rest_crud` generan **todos los endpoints** de una entidad.

🔑 CLAVES & ACLARACIONES

- Convención: GET /<e>/ (lista) · GET /<e>/{id} · POST · PUT · DELETE · GET /<e>/schema.json (ETag).
- Códigos→HTTP: 400 / 401 / 403 / 404 / 409 / 422 / 500.
- VALIDATION_ERROR lleva `location` (viene de Pydantic). (§7.3.2)

Descubrimiento + cliente genérico

El frontend pregunta qué entidades hay, no las codifica.

🔊 QUÉ DECIR

- ▶ `/api/sys/entities` lista las entidades CRUD con sus capacidades; `/api/sys/discovery`, servicios externos (Traefik); `/api/sys/schemas`, el bundle.
- ▶ En el front, `EntityClient` da un cliente **tipado por path**:
`api.for<Role>('/roles')` → `list`, `get`, `create`, `update`, `schema`.
- ▶ `SchemaService` cachea los esquemas; un interceptor opt-in convierte fallos HTTP en envoltorios.
- ▶ Convive con el `BackendService` heredado (~272 métodos) sin romperlo: **migración progresiva**.

🔑 CLAVES & ACLARACIONES

- Capacidades por entidad: `list` · `get` · `create` · `update` · `delete` · `schema`.
- Capa **aditiva** → no rompe código en producción. (§7.3.2.3–.4)
- Esto es lo que hace el sistema **autodescriptivo**.

→ El núcleo de la aportación: **enriquecer el esquema**.

Backend · enriquecimiento del esquema

Heurísticas x- por columna.*

🔊 QUÉ DECIR

- ▶ `sqlalchemy_to_pydantic` se amplió para **inyectar pistas de presentación por columna** en `json_schema_extra`.
- ▶ **Boolean** → **checkbox** · **DateTime** → **datepicker (con hora)** · **String largo** → **textarea** · **ForeignKey** → **select-lazy-loading** con su `x-options-endpoint`.
- ▶ Es **heurística**: los **overrides manuales** en `column.info["ui"]` **siempre ganan**.
- ▶ Las pistas viajan **dentro del propio esquema** → sobreviven a cualquier serialización o cacheo.

🔑 CLAVES & ACLARACIONES

- Extensiones reconocidas: `x-formly-type`, `x-ui-widget`, `x-options-endpoint`, `x-placeholder`, `x-formly-props`, `x-hide-when`, `x-required-when`, `x-disabled-when`, `x-discriminator`, `x-relationships`, `x-foreign-key`. (§7.3.3.1)

→ Y en el frontend: el conversor y el componente.

Frontend · conversor y componente

Una etiqueta, un formulario.

🗣️ QUÉ DECIR

- ▶ `schemaToFormly` es un **conversor puro**: JSON Schema → configuración Formly.
- ▶ Resuelve `$ref` **recursivamente**, mapea **object**→**fieldGroup** y **array**→**repeat**, e interpreta `x-formly-type`, `x-ui-widget`, `x-hide-when`, `x-required-when`.
- ▶ `<ngt-dynamic-form>` es la cara visible: con solo **entityPath** descubre la entidad, pide el esquema, lo convierte, monta el formulario, valida y envía vía `EntityClient`.
- ▶ **Tres modos**: create, edit, view.

🔑 CLAVES & ACLARACIONES

- Validadores derivados de `minLength`, `maxLength`, `pattern`, `format`.
- El conversor **no depende de la UI**.
- Composición interna: `DiscoveryService` → `SchemaService` → `schemaToFormly` → `EntityClient`. (§7.3.3.3–.4)

Generación automática de esquemas

Bundle versionado, polimorfismo y relaciones.

🔊 QUÉ DECIR

- ▶ Cada fábrica CRUD **registra la entidad en un registry global**; `build_schema_bundle` construye un documento agregado con todas las entidades.
- ▶ `compute_etag` calcula un **hash SHA-256 truncado**; el bundle y cada `schema.json` responden **304** ante `If-None-Match` coincidente (caché agresiva).
- ▶ **Dos piezas de cierre**: el polimorfismo de `FunctionalObject` se refleja como `oneOf` + `x-discriminator` sobre `object_type_id`; los `relationship()` del ORM, como `x-relationships`.

🔑 CLAVES & ACLARACIONES

- Decisión: **incrustar las variantes** (no `$ref`) → bundle **autocontenido y determinista** (ETag estable entre llamadas idénticas).
- Habilita **codegen externo**: tipos TypeScript, validadores, documentación, espejos de microservicios. (§7.3.4)

Pruebas automatizadas

Cobertura por incremento, backend y frontend.

🗣️ QUÉ DECIR

- ▶ **Backend (pytest)**: envoltorio, autenticación, heurística de pistas de UI por columna, y el **bundle con sus invariantes**.
- ▶ **Frontend (Jasmine)**: cobertura del **conversor a Formly** (tipos, overrides `x-*`, `$ref`, expresiones reactivas, modo view, raíz polimórfica).
- ▶ Estrategia: **unit tests** de las piezas puras (heurística, conversor) + **integración** de la API sobre PostgreSQL — cubre los puntos donde un cambio del modelo podría romper la cadena.

🔑 CLAVES & ACLARACIONES

- Invariantes del bundle: versión estable · round-trip de ETag (304) · sensibilidad del hash · oneOf+x-discriminator · x-relationships.
- Suites: `test_envelope.py`, `test_auth.py`, `test_form_schema.py`, `test_schema_bundle.py`, `schema-to-formly.spec.ts` (§7.4)

Antes / después

El resultado más significativo es cualitativo.

🗣️ QUÉ DECIR

- ▶ Exponer y editar una entidad nueva pasa de: **router a mano + esquemas Pydantic + métodos de servicio + HTML campo a campo...**
- ▶ ...a: **declarar el modelo ORM** y dejar que el framework **derive el resto**.
- ▶ **El esfuerzo deja de crecer** con el número de entidades y con la frecuencia de cambios del modelo.
- ▶ Los cinco objetivos específicos están **completados y respaldados por pruebas**.

🔑 CLAVES & ACLARACIONES

- Tabla antes→después: CRUD (router a mano → `make_simple_rest_crud`) · validación (Pydantic a mano → derivada del ORM) · contrato (ad hoc → `ResponseEnvelope`) · cliente (método en `BackendService` → `EntityClient.for(path)`) · formulario (HTML por campo → `<ngt-dynamic-form>`). (§8.2.1)

Demo en vivo

Una entidad nueva: del modelo ORM al formulario renderizado.

🗣️ QUÉ DECIR

- ▶ 1) Declaro el modelo: `class MiEntidad(orm_base): ...`
- ▶ 2) CRUD + contrato + esquema:
`make_simple_rest_crud(MiEntidad, "/mi-entidad")`.
- ▶ 3) Formulario generado: `<ngt-dynamic-form
entityPath="/mi-entidad">`.
- ▶ **Resultado: cero líneas de pegamento entre capas.**

🔑 CLAVES & ACLARACIONES

- ⚠ Antes de empezar: servidor levantado, base de datos lista, navegador en la vista correcta.
- Plan B si algo falla: tener a mano una captura/figura del flujo y narrarlo. (§8.3)
- Avisar al tribunal de lo que van a ver antes de hacerlo.

Conclusiones y líneas futuras

El modelo ORM como única fuente de verdad: el resto se deriva.

🗣️ QUÉ DECIR

- ▶ **Cuatro líneas de continuación:**
- ▶ 1) Codegen de tipos TypeScript y documentación automática desde el bundle.
- ▶ 2) Migrar progresivamente el `BackendService` heredado para que delegue en `EntityClient`.
- ▶ 3) Soportar `oneOf` / `anyOf` en el conversor a Formly + overrides por path.
- ▶ 4) Aplicar la misma filosofía guiada por esquema a las **vistas de listado/tabla**, no solo a los formularios.

🔑 CLAVES & ACLARACIONES

- Agradecer a **Rafael Juan Nebot Medina** y a **María Dolores Afonso Suárez**.
- Frase de cierre: «**Muchas gracias por su atención. Quedo a disposición del tribunal.**»
- Lema: «El modelo ORM como única fuente de verdad: el resto se deriva.» — ULPGC · EII · Feb 2026 · ITC.